COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	7/29/26	**DATE SUBMITTED:**	8/4/26
NAME:	Condino, Niel Vincent B.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY 4.1 STACKS

1. PROCEDURE OUTPUT

```cpp
#include <iostream>
#include <stack>

int main(){

    std::stack<int> stack;
    std::cout << "Testing stack STL\n";

    //check if empty
    std::cout << "Bruv, Is them stack empty innit? " << stack.empty() << std::endl;

    //push
    stack.push(1);
    std::cout << "Bruv, the thing at the top of the stack is " << stack.top() << std::endl;
    stack.push(10);
    std::cout << "Bruv, the thing at the top of the stack is " << stack.top() << std::endl;
    stack.push(100);
    std::cout << "Bruv, the thing at the top of the stack is " << stack.top() << std::endl;
    stack.push(1000);
    std::cout << "Bruv, the thing at the top of the stack is " << stack.top() << std::endl;
    stack.pop();
    std::cout << "Bruv, the thing at the top of the stack is " << stack.top() << std::endl;
    std::cout << "Mate, the stack is " << stack.size() << " elements big\n";

    std::cout << "Bruv, Is them stack empty innit? " << stack.empty() << std::endl;

    std::cout << "Ey' mate, im gooing to emplace 25 at top of the stack" << std::endl;
    stack.emplace(25);
    std::cout << "Bruv, the thing at the top of the stack is " << stack.top() << std::endl;
    std::cout << "Mate, the stack is " << stack.size() << " elements big\n";

    std::cout << "I'll create new stack mate\n";
    std::stack<int> stackJr;
    stackJr.push(2);
    std::cout << "Bruv, the thing at the top of the stackJr is " << stackJr.top() << std::endl;
    stackJr.push(20);
    std::cout << "Bruv, the thing at the top of the stackJr is " << stackJr.top() << std::endl;
    stackJr.push(200);
    std::cout << "Bruv, the thing at the top of the stackJr is " << stackJr.top() << std::endl;
    stackJr.push(2000);
    std::cout << "Bruv, the thing at the top of the stackJr is " << stackJr.top() << std::endl;
    std::cout << "Mate, the stackJr is " << stackJr.size() << " elements big\n";

    std::cout << "Buddy, im going to swap them stackJr\n";

    stack.swap(stackJr);
```

```
    std::cout << "Bruv, the thing at the top of the stack is " << stack.top() << std::endl;
    std::cout << "Mate, the stack is " << stack.size() << " elements big\n";

    std::cout << "Bruv, the thing at the top of the stackJr is " << stackJr.top() << std::endl;
    std::cout << "Mate, the stackJr is " << stackJr.size() << " elements big\n";

    return 0;
}
```

```
Testing stack STL
Bruv, Is them stack empty innit? 1
Bruv, the thing at the top of the stack is 1
Bruv, the thing at the top of the stack is 10
Bruv, the thing at the top of the stack is 100
Bruv, the thing at the top of the stack is 1000
Bruv, the thing at the top of the stack is 100
Mate, the stack is 3 elements big
Bruv, Is them stack empty innit? 0
Ey' mate, im gooing to emplace 25 at top of the stack
Bruv, the thing at the top of the stack is 25
Mate, the stack is 4 elements big
I'll create new stack mate
Bruv, the thing at the top of the stackJr is 2
Bruv, the thing at the top of the stackJr is 20
Bruv, the thing at the top of the stackJr is 200
Bruv, the thing at the top of the stackJr is 2000
Mate, the stackJr is 4 elements big
Buddy, im going to swap them stackJr
Bruv, the thing at the top of the stack is 2000
Mate, the stack is 4 elements big
Bruv, the thing at the top of the stackJr is 25
Mate, the stackJr is 4 elements big
```

Figure 1: ILOA code and output

In this part of the procedure, we use the stack objects from STL to learn how it works. From what I observed and learned, it implements a last-in, first-out model. The methods for it reflect this approach. The push() function puts a given value and puts it at the top of the stack. The pop() method is the reverse of this; it removes or “pops” the element at the top of the stack. The top() method is used to display the element currently at the top of the list. To check if it's empty, use the empty() method; it returns a boolean indicating whether the stack is empty. The size() method is used to check the number of elements inside the stack. The emplace() method has the same functionality as push(); however, push() creates a copy and then pushes it into the stack, while emplace() creates the element at the top of the stack. The last method used is the swap() method, used to swap elements of 2 different stack objects. The stack object from STL provides much easier accessibility to this model because if you just need a “general” stack, this library already provides you with a stack with all different methods.

LOB1 > main.cpp > displayStack()

```cpp
#include <iostream>

//global declarations
#define maxCap 10
int stackArr[maxCap];
int topArr = -1,newData;

//prototype function
void push();
void pop();
void top();
bool isEmpty();
bool isFull();
void displayStack();

int main(){
    bool continueLoop = true;

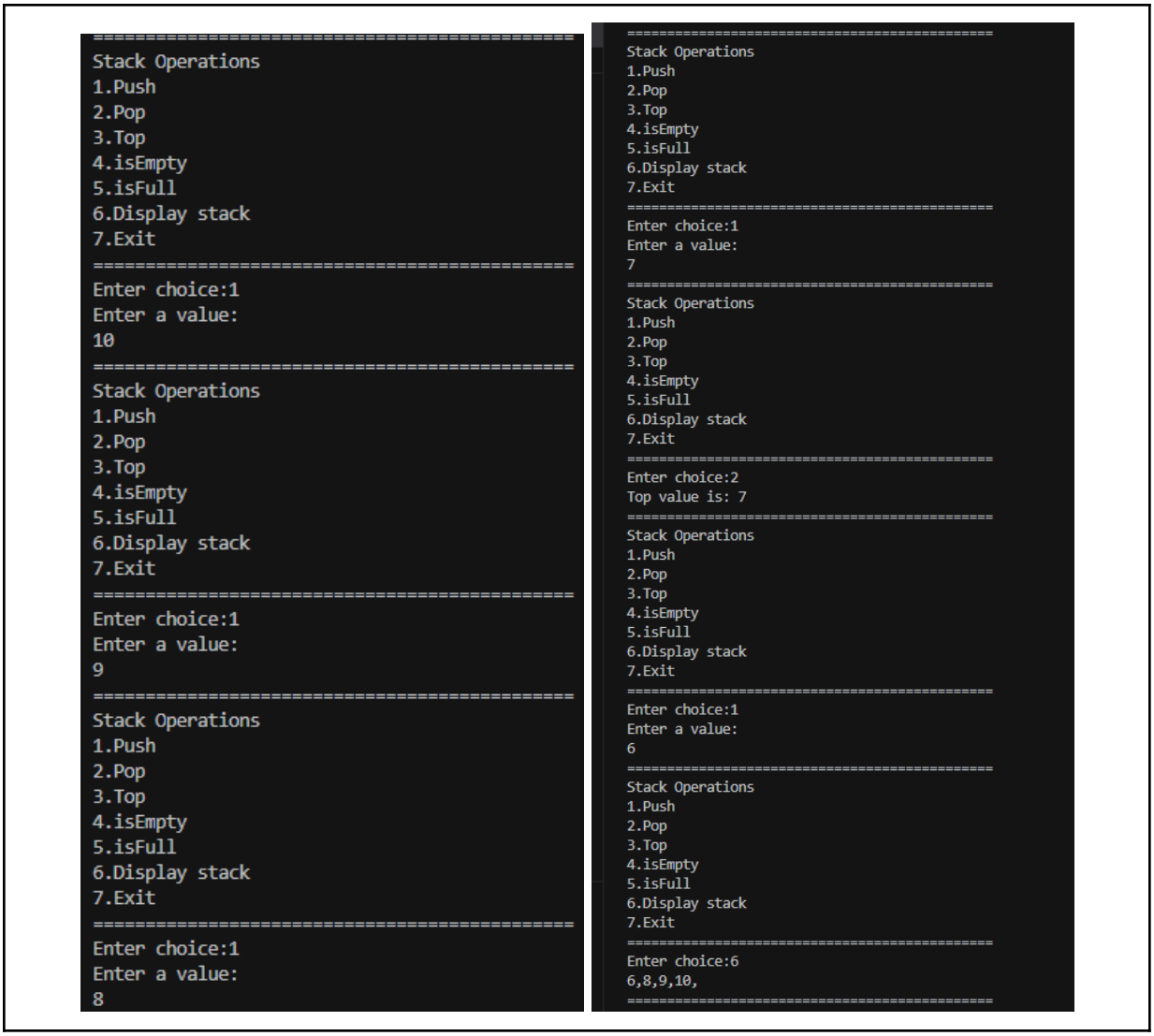
    while (continueLoop){
        int choice = 0;
        std::cout << "=============================================\n";
        std::cout << "Stack Operations\n";
        std::cout << "1.Push\n2.Pop\n3.Top\n4.isEmpty\n5.isFull\n6.Display stack\n7.Exit";
        std::cout << "\n=============================================\n";

        std::cout << "Enter choice:";
        std::cin >> choice;

        switch (choice){
            case 1:
            {
                push();
                break;
            }
            case 2:{
                pop();
                break;
            }
            case 3:{top();
            break;}
            case 4: std::cout << "Is Empty: " << isEmpty() << std::endl;
            break;
            case 5: std:: cout << "Is Full : " << isFull() << std::endl;
            break;
```

```
45                  case 6: displayStack();
46                  case 7: continueLoop = false;
47                  break;
48                  default:std::cout << "invalid choice";break;
49              }
50          }
51          //main driver
52          return 0;
53      }
54
55      //definiton
56
57      bool isEmpty(){
58          if (topArr == -1){
59              return true;
60          }
61          return false;
62      }
63
64      bool isFull(){
65          if (topArr == maxCap - 1) return true;
66          return false;
67      }
68
69      void push(){
70          //err check
71          if (isFull()){
72              std::cout << "stack overflow\n";
73              return;
74          }
75          //pushing
76          std::cout << "Enter a value: \n";
77          std::cin >> newData;
78          //insert in the stack
79          stackArr[++topArr] = newData;
80      }
81
82      void pop(){
83          //err check
84          if (isEmpty()){
85              std::cout << "stack underflow\n";
86              return;
```

```
87      }
88
89      //display the popped value
90      top();
91
92      //decrement the top value from the class
93      topArr--;
94  }
95
96  void top(){
97      //err
98      if (isEmpty()){
99          std::cout << "no stack elements\n";
100         return;
101     }
102     std::cout << "Top value is: " << stackArr[topArr] << std::endl;
103 }
104
105 void displayStack(){
106     for (int i = topArr; i >= 0;i--){
107         std::cout << stackArr[i] << ",";
108     }
109     std::cout << '\n';
110 }
```



```
==========================================
Stack Operations
1.Push
2.Pop
3.Top
4.isEmpty
5.isFull
6.Display stack
7.Exit
==========================================
Enter choice:1
Enter a value:
10
==========================================
Stack Operations
1.Push
2.Pop
3.Top
4.isEmpty
5.isFull
6.Display stack
7.Exit
==========================================
Enter choice:1
Enter a value:
9
==========================================
Stack Operations
1.Push
2.Pop
3.Top
4.isEmpty
5.isFull
6.Display stack
7.Exit
==========================================
Enter choice:1
Enter a value:
8
==========================================
Stack Operations
1.Push
2.Pop
3.Top
4.isEmpty
5.isFull
6.Display stack
7.Exit
==========================================
Enter choice:1
Enter a value:
7
==========================================
Stack Operations
1.Push
2.Pop
3.Top
4.isEmpty
5.isFull
6.Display stack
7.Exit
==========================================
Enter choice:2
Top value is: 7
==========================================
Stack Operations
1.Push
2.Pop
3.Top
4.isEmpty
5.isFull
6.Display stack
7.Exit
==========================================
Enter choice:1
Enter a value:
6
==========================================
Stack Operations
1.Push
2.Pop
3.Top
4.isEmpty
5.isFull
6.Display stack
7.Exit
==========================================
Enter choice:6
6,8,9,10,
==========================================
```

Figure 2: ILOB code and output.

In this part of the activity, we learned to create a custom stack using an array. This stack is considered “static” as the size is already declared at compile-time. To properly create a stack in this method, different global variables are declared, like the one storing the current index of the stack. We also created the different generic methods such as push() and pop(), which, as their names suggest, push an element to the stack and “pop” or remove the top element. Some functions just check or display the elements inside the stack, which are top(), isEmpty(), and isFull(). I also created a function named displayStack() which displays all the elements of the stack from top to bottom. To check these functions, they are called in the main function, where the user can choose to modify or just see the stack. As seen in the outputs, the code is implemented and works correctly.

```cpp

//class node that contains data and the next
template <typename T>
class Node{
    public:
    T data;     // VALUE THAT WE NEED STORE
    Node* next; // POINTER TO THE NEXT NODE
};

//global pointer for the top of the stack
template <typename T>
Node<T> *head = nullptr;

//operations for the stack

// push
template <typename T>
void push(T newData){
    //create a new node and assign the newData
    Node<T>* newNode = new Node<T>;
    newNode->data= newData;

    //new node points to current top, then becomes the new top
    newNode->next = head<T>;
    head<T> = newNode;
}

// isEmpty
template<typename T>
bool isEmpty(){
    return head <T> == nullptr;
}

// pop
template<typename T>
T pop(){
    //check if the stack is empty:
    if(isEmpty<T>()){
        std::cout<< "stack underflow. "<<std::endl;
        return T();
    }

    //save the current top node
    Node<T>* temp = head<T>;
    //store its data
```

```
47      //save the current top node
48      Node<T>* temp = head<T>;
49      //store its data
50      T tempVal = temp->data;
51
52      // move the head to the next node
53      head<T> = head<T>->next;
54      //free the removed node
55      delete temp;
56      //return the removed value
57      return tempVal;
58  }
59
60  // top
61  template<typename T>
62  T Top(){
63      // error catching:
64      if(isEmpty<T>()){
65          std::cout<<"Stack underflow "<<std::endl;
66          return T();
67      }
68      // display the Top node
69      return head<T>->data;
70      std::cout<<"Top of Stack: "<<head<T>->data <<std::endl;
71  }
72
73  //display All
74  template <typename T>
75  void displayALL(){
76      if (isEmpty<T>()){
77          std::cout << "stack undeflow\n";
78          return;
79      }
80      std::cout<< "Displaying the stack: \n";
81      Node<T>* currentElem = head<T>;
82      while(currentElem != nullptr){
83          std::cout << currentElem->data << "\n";
84          currentElem = currentElem->next;
85      }
86      std::cout << "\n";
87  }
88
89  #endif
```

Figure 3: Stack for Linked list module

The code is about the stack implementation using the linked list data structure. The isEmpty() function checks if there is a list by checking if the head pointer is pointing at a real address. Pushing data into the stack just creates a new node with the new linked node holding that data and places it as the new head. The top() function peaks at the topmost data by checking the head of the list. Popping's algorithm first checks whether there is no list; if there is, it takes the current head's address and stores it in a temporary variable. The current head pointer is shifted to the next element, and the last head's memory is then freed, and the data is returned. If it triggers a stack underflow, it prints out a message and returns a default "void" value for each type. The displayAll() function just traverses the stacks and prints out data that each node stores.

```cpp
#include<iostream>
#include "stackList.h"

int main(){

    std::cout<< "--- PUSHING 'n' 'i' 'e' 'l' ONTO THE STACK ---- \n";
    //push
    push(newData: 'l');
    push(newData: 'e');
    push(newData: 'i');
    push(newData: 'N');

    //display the current top
    Top<char>(); // Top should be 'e'
    displayALL<char>();
    std::cout<<"\n ------ Poppping all the elements ----- \n";

    while(!isEmpty<char>()){
        std::cout<<" Popped: "<< pop<char>() <<std::endl;
    }

    std::cout<<"\n ------ Testing the Underflow ----- \n";
    pop<char>();

    return 0;
}
```

Figure 4:Stack List main driver

This code just utilizes the code from the last figure, which is the linked list implementation of a stack. It first pushes the different characters c,p, and e. It then checks the top and checks all the data of the nodes. Next, it implements a while loop that pops every node in the linked list. Lastly, it checks what will happen if the stack underflows.

```
~/Downloads/HOA/HW#4
❯ cd "/home/nielc/Downloads/HOA/HW#4/" && g++ -std=c++20 list_stacks.cpp -o list_stacks -lncurses && "/home/nielc/Downloads/HOA/HW#4/"list_stacks
--- PUSHING 'n' 'i' 'e' 'l' ONTO THE STACK ----
Displaying the stack:
N
i
e
l


 ------ Poppping all the elements -----
 Popped: N
 Popped: i
 Popped: e
 Popped: l

 ------ Testing the Underflow -----
stack underflow.
```

Figure 5: Figure 4 code output

The output above shows the output of the code in Figure 4. When displaying each data on the stack, it goes in LIFO order. The e character prints out first, while c prints out last. Popping each element also follows the LIFO order. Lastly, the stack underflow is handled without an error due to the function returning a default “void” value.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

```cpp
#include <iostream>
#include <string>
#include <unordered_map>

//global declarations
#define maxCap 50
char stackArr[maxCap];
int topArr = -1;

std::unordered_map<char, char> symbols ={
//Closing symbols
[0]={']','['},
[1]={')','('},
[2]={'}','{'},
//Opening symbols
[3]={'{','\0'},
[4]={'[','\0'},
[5]={'(','\0'},
};

//prototype function
void push(char givenData);
void pop();
char top();
bool isEmpty();
bool isFull();
void displayStack();
void balanceSign(std::string input);

//main driver
int main(){
    balanceSign(input: "(A+B)+(C-D)");
    //balanceSign("((A+B)+(C-D)");
    //balanceSign("(A+B)+[C-D])");
    //balanceSign("((A+B]+[C-D]}");
    return 0;
}

void balanceSign(std::string input){
    for (auto character: char: input){
        if (symbols.contains(x: character)){
            //check if opening symbol
            if (symbols.at(k: character) == '\0'){
                push(givenData: character);
            }
            else{
                if (isEmpty()){
                    std::cout << input << " : " << "Error (Not Valid): Stack is empty\n";
                    return;
```

```
                }
                else if (top() == symbols.at(k: character)){
                    pop();
                }
                else{
                    std::cout << input << " : " << "Error (Not Valid): closing "
                    "symbol does not match opening symbol\n";
                    return;
                }
            }
        }
    }

    if (!isEmpty()){
        std::cout << input << " : " << "Error (Not Valid): stack has not been emptied\n";
    }
    else{
        std::cout << "Checking complete: "<< input << " is valid\n";
    }
}

//definiton

bool isEmpty(){
    if (topArr == -1){
        return true;
    }
    return false;
}

bool isFull(){
    if (topArr == maxCap - 1) return true;
    return false;
}

void push(char givenData){
    //err check
    if (isFull()){
        std::cout << "stack overflow\n";
        return;
    }
    //insert in the stack
    stackArr[++topArr] = givenData;
}

void pop(){
    //err check
    if (isEmpty()){
        std::cout << "stack underflow\n";
        return;
    }

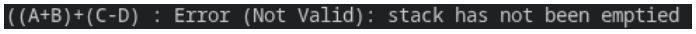
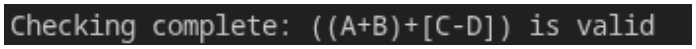
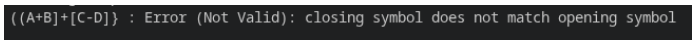
    //decrement the top value from the class
    topArr--;
}

char top(){
    //err
    if (isEmpty()){
        std::cout << "no stack elements\n";
        return '\0';
    }
    return stackArr[topArr];
}

void displayStack(){
    for (int i = topArr; i >= 0; i--){
        std::cout << stackArr[i] << ",";
    }
    std::cout << '\n';
}
```

Figure 6:Balancing of symbols, array stacks edition

The algorithm for the main function, balanceSign(), works on a series of steps. It first loops through the strings, checking each character. It checks if each character is inside the map where the closing and their corresponding opening symbols are stored. It then checks if the symbol is an opening symbol. If it is, the character is pushed into the stack. If it is not, it checks if the stack is empty; if it is, a closing symbol appears before an opening symbol. This prints out an error. If not, it checks if the top of the stack is the same opening symbol as the mapped value on the corresponding closing symbol from the map. If not, it prints out an error. Next, it checks if the stack has not been emptied. If it is, then it prints out an error. If it is not, it prints out a message saying that the expression is valid.

Expression	Valid? (Y/N)	Output (Console Screenshot)	Analysis
(A+B)+(C-D)	Y		Expression is valid because all open parentheses have been closed
((A+B)+(C-D)	N		Expression is not valid because the first open parenthesis has not been closed
((A+B)+[C-D])	Y		Expression is valid because all open parentheses have been closed
((A+B]+[C-D]}	N		Expression is not valid because the last closing symbol does not

```cpp
#include <iostream>
#include <string>
#include <unordered_map>
#include "stackList.h"

std::unordered_map<char, char> symbols ={
//Closing symbols
[0]={']','['},
[1]={')','('},
[2]={'}','{'},
//Opening symbols
[3]={'{','\0'},
[4]={'[','\0'},
[5]={'(','\0'},
};

void balanceSign(std::string input);

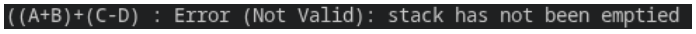
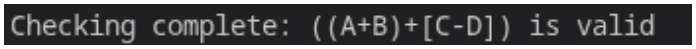
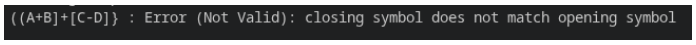
//main driver
int main(){
    //balanceSign("(A+B)+(C-D)");
    //balanceSign("((A+B)+(C-D)");
    //balanceSign("(A+B)+[C-D])");
    balanceSign(input: "((A+B]+[C-D]}");
    return 0;
}

void balanceSign(std::string input){
    for (auto character: char: input){
        if (symbols.contains(x: character)){
            //check if opening symbol
            if (symbols.at(k: character) == '\0'){
                push(newData: character);
            }
            else{
                if (isEmpty<char>()){
                    std::cout << input << " : " << "Error (Not Valid): Stack is empty\n";
                    return;
                }
                else if (Top<char>() == symbols.at(k: character)){
                    pop<char>();
                }
                else{
                    std::cout << input << " : " << "Error (Not Valid): closing "
                    "symbol does not match opening symbol\n";
                    return;
                }
            }
        }
    }

    if (!isEmpty<char>()){
        std::cout << input << " : " << "Error (Not Valid): stack has not been emptied\n";
    }
    else{
        std::cout << "Checking complete: "<< input << " is valid\n";
    }
}
```

Figure 7:Balancing of symbols, list stacks edition

This code shares the algorithm from Figure 6. This, however, uses the linked list implementation, which is already provided by the module created in the procedure part of the laboratory. While the algorithm for the program remains the same, the calling of methods for the stack object is different. The methods were called using the syntax “function<datatype>()”. While calling the function is different, the output remains the same as in Figure 6.

Expression	Valid? (Y/N)	Output (Console Screenshot)	Analysis
(A+B)+(C-D)	Y		Expression is valid because all open parentheses have been closed
((A+B)+(C-D)	N		Expression is not valid because the first open parenthesis has not been closed
((A+B)+[C-D])	Y		Expression is valid because all open parentheses have been closed
((A+B]+[C-D]}	N		Expression is not valid because the last closing symbol does not

```cpp
#include <iostream>
#include <string>
#include <unordered_map>
#include <stack>

std::unordered_map<char, char> symbols ={
//Closing symbols
[0]={']','['},
[1]={')','('},
[2]={'}','{'},
//Opening symbols
[3]={'{','\0'},
[4]={'[','\0'},
[5]={'(','\0'},
};

void balanceSign(std::string input);

std::stack<char> stack;

//main driver
int main(){
    balanceSign(input: "(A+B)+(C-D)");
    //balanceSign("((A+B)+(C-D)");
    //balanceSign("(A+B)+[C-D])");
    //balanceSign("((A+B]+[C-D]}");
    return 0;
}

void balanceSign(std::string input){
    for (auto character: char: input){
        if (symbols.contains(x: character)){
            //check if opening symbol
            if (symbols.at(k: character) == '\0'){
                stack.push(x: character);
            }
            else{
                if (stack.empty()){
                    std::cout << input << " : " << "Error (Not Valid): Stack is empty\n";
                    return;
                }
                else if (stack.top() == symbols.at(k: character)){
                    stack.pop();
                }
                else{
                    std::cout << input << " : " << "Error (Not Valid): closing "
                    "symbol does not match opening symbol\n";
                    return;
                }
```

```
50              }
51          }
52      }
53
54      if (!stack.empty()){
55          std::cout << input << " : " << "Error (Not Valid): stack has not been emptied\n";
56      }
57      else{
58          std::cout << "Checking complete: "<< input << " is valid\n";
59      }
60  }
```

Figure 8:Balancing of symbols, STL stacks edition

This code shares the algorithm from Figure 6. This, however, uses the STL library available for C++, which gives access to the stack object. While the algorithm for the program remains the same, the calling of methods for the stack object is different. The methods were called using the syntax “stack. function()”. While calling the function is different, the output remains the same as in Figure 6.

Expression	Valid? (Y/N)	Output (Console Screenshot)	Analysis
(A+B)+(C-D)	Y	`Checking complete: (A+B)+(C-D) is valid`	Expression is valid because all open parentheses have been closed
((A+B)+(C-D)	N	`((A+B)+(C-D) : Error (Not Valid): stack has not been emptied`	Expression is not valid because the first open parenthesis has not been closed

((A+B)+[C-D])	Y	`Checking complete: ((A+B)+[C-D]) is valid`	Expression is valid because all open parentheses have been closed
((A+B]+[C-D]}	N	`((A+B]+[C-D]} : Error (Not Valid): closing symbol does not match opening symbol`	Expression is not valid because the last closing symbol does not

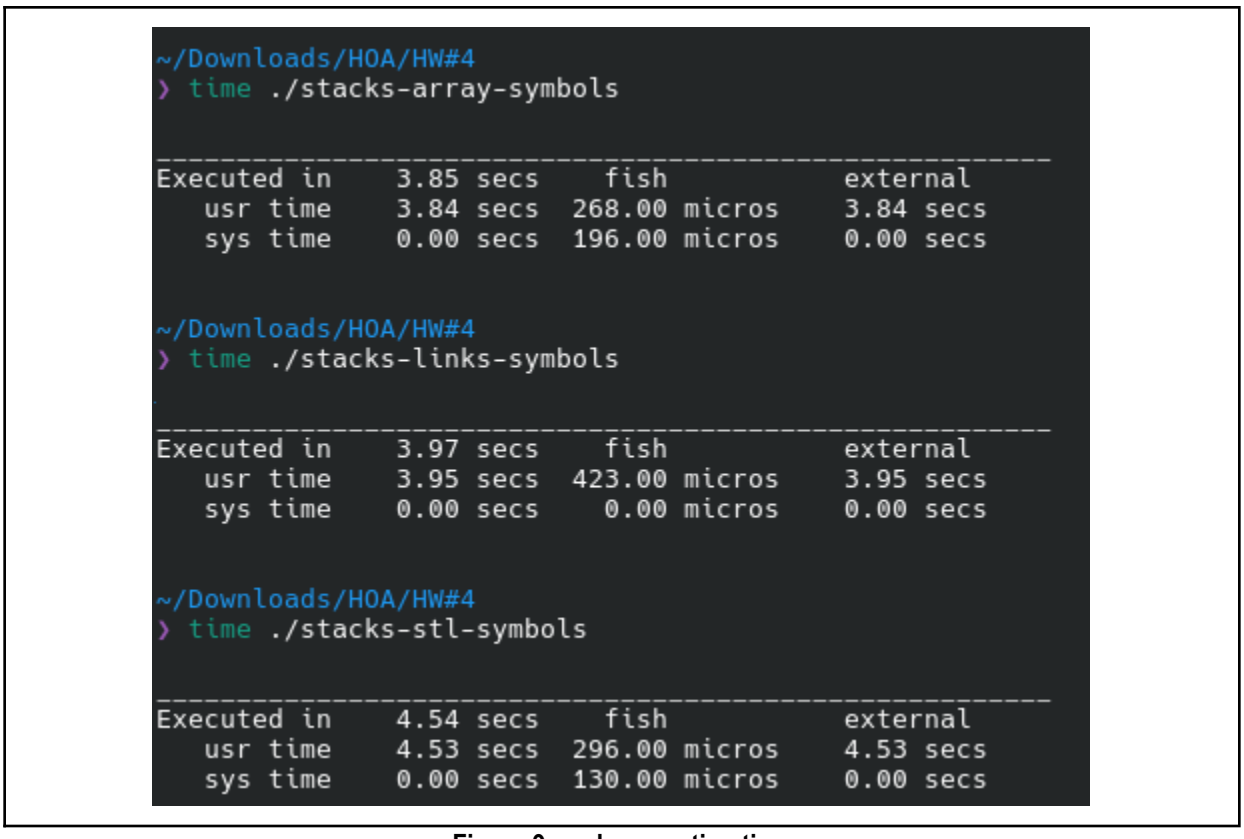


Figure 9: code execution time

Tool Analysis:

In an array-based stack, elements occupy contiguous memory addresses, allowing the CPU to cache the neighboring data, which results in making operations practically faster. Static arrays have a fixed size. An overflow can happen if the number of data exceeds the array's size. The memory it also has is fixed, meaning most of the time when the stack is not full, much of the memory is wasted. In contrast, a list implementation stack features a dynamic size, meaning the size is not hard-coded at compile time, so there is little risk of overflow unless the memory is limited. However, because the nodes are distributed throughout the heap and are connected solely by their pointer members, this implementation has a larger memory footprint due to the need to store these additional pointers. Not being contiguous also means the CPU has to access the memory more often instead of relying on the cache, making it slower in practice.

To test out their execution time, I performed a for loop that reiterates a million times. As the output shows, as predicted in theory, the array implementation comes first, followed by the linked list, and lastly by STL.

Conclusion:

In this activity, I was able to create and implement the concept of a stack. A stack uses the last-in, first-out model, meaning the last one pushed into the stack is the first one to get popped. I also learned the different functions relating to stacks, such as push(), pop(), top(), or peek(), and the isFull() and isEmpty() functions. These serve different but integral functions for modifying or checking the stack. I am also able to implement this concept in different implementations such as the array-based and the linked list-based. Each of the implementations have their advantages and tradeoffs, such as array-based being CPU-friendly while not being memory-friendly because of its fixed size, where most of the time it is not fully used. Linked list im-plementation in the other hand, offers much better memory usage for the number of elements. However, it features higher memory count per element as it needs to hold the memory for the data and the pointer that links it to the next node. It is also not CPU-friendly as it will need to rely on the much slower memory instead of utilizing the cache. Overall, I aim to use my learnings on the concept and implementation of stacks in my future programming when the problem needs the concept of a stack for implementing my solution.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Only used to fix bad grammar**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

COMPUTER ENGINEERING

QUEZON CITY

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.